

Learning vLLM: The 20% That Matters

1 What vLLM Is and When to Reach for It

vLLM is an open-source **inference and serving engine** for LLMs, originally from UC Berkeley (the PagedAttention paper, 2023). It is now the de facto standard for self-hosted, high-throughput LLM serving. The point of vLLM is *speed at scale*: it delivers many times the aggregate throughput of simpler tools like Ollama on multi-user workloads.

Where it fits with the other tools:

- The *API-calls* document was about calling *someone else's* hosted model. vLLM is how you **host your own** model behind that same kind of API.
- It serves the Hugging Face models you learned to load, and the LoRA adapters you produced in the *fine-tuning* document.
- It exposes an **OpenAI-compatible** API, so RAG, LlamaIndex, and LangGraph can point at it by just changing the base URL.

Reality check: vLLM is GPU-focused. If you do not have a GPU, rent one ad-hoc (cloud A100/H100) or test with a very small model. Don't expect it to run well on a laptop CPU.

2 Why It Is Fast (Two Core Ideas)

You only need to understand two innovations:

- **PagedAttention** — the KV cache (the memory that grows as a model generates tokens) is managed in small “pages” like operating-system virtual memory. This eliminates the huge memory waste of reserving a fixed block per request, so you fit far more concurrent requests in the same VRAM.
- **Continuous batching** — instead of waiting to fill a fixed batch, vLLM adds and removes requests from the running batch every step. New requests start immediately; finished ones free their slot at once. This keeps the GPU busy and is the main reason throughput is so high under concurrency.

You rarely touch these directly — they are why the same hardware serves many more users.

3 Setup

```
1 pip install vllm # needs a CUDA GPU + matching drivers
```

4 Mode 1 — Offline Batch Inference

For processing a big list of prompts at once (evaluation, data generation), use the Python LLM class. vLLM batches internally — you just hand it all the prompts.

```
1 from vllm import LLM, SamplingParams
2
3 llm = LLM(
4     model="meta-llama/Llama-3-8B-Instruct",
5     gpu_memory_utilization=0.9,      # fraction of VRAM to use
6     max_model_len=4096,              # max context length
7 )
8
9 sampling = SamplingParams(temperature=0.7, top_p=0.95, max_tokens=512)
10
11 prompts = ["Explain RAG in one sentence.", "What is QLoRA?"]
12 outputs = llm.generate(prompts, sampling)  # batched automatically
13
14 for out in outputs:
15     print(out.prompt)
16     print(out.outputs[0].text)             # the generated text
17     print(len(out.outputs[0].token_ids))   # number of generated tokens
```

Output shape: `generate` returns one result per prompt; each has `.outputs` (a list, one per sampled sequence), and `.outputs[0].text` is the completion. Do not manually chunk prompts — passing them all in one call is what lets vLLM batch efficiently.

5 SamplingParams (The Generation Knobs)

- `temperature` — randomness; 0 = deterministic, higher = more varied.
- `top_p` — nucleus sampling; keep the smallest set of tokens summing to this probability.
- `max_tokens` — cap on generated length.
- `stop` — list of strings that end generation, e.g. ["</s>", "\n\n"].
- `n` — number of completions to return per prompt.

6 Mode 2 — Online Serving (The Main Use)

This is what you will use most: one command starts a production server.

```
1 vllm serve meta-llama/Llama-3-8B-Instruct \
2   --host 0.0.0.0 --port 8000 \
3   --gpu-memory-utilization 0.9 \
4   --max-model-len 8192
```

This launches an **OpenAI-compatible** HTTP server. That single fact is the payoff: any code written for the OpenAI API works against it.

7 Calling Your Server

Reuse exactly the patterns from the API-calls document — just change the base URL and use a dummy key.

```
1 from openai import OpenAI
2
3 client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")
4
5 resp = client.chat.completions.create(
6     model="meta-llama/Llama-3-8B-Instruct",
7     messages=[{"role": "user", "content": "Hello!"}],
8     temperature=0.7,
9 )
10 print(resp.choices[0].message.content)
```

Or with plain requests / curl against `http://localhost:8000/v1/chat/completions`. Streaming, tool calling, and the usual fields are supported.

8 Key Serving Flags

- `--gpu-memory-utilization 0.9` — how much VRAM vLLM may use (higher = more room for the KV cache = more concurrency).
- `--max-model-len` — max context length per request.
- `--max-num-seqs` — max concurrent sequences in a batch.
- `--tensor-parallel-size N` — split the model across N GPUs.
- `--quantization <method>` — run a quantized model (see below).
- `--enable-prefix-caching` — reuse computation for shared prompt prefixes.

9 Multi-GPU: Tensor Parallelism

A model too big for one GPU is split across several with one flag:

```
1 # 70B model across 4 GPUs
2 vllm serve meta-llama/Llama-2-70b-hf --tensor-parallel-size 4 --gpu-memory
   -utilization 0.9
```

`tensor_parallel_size` shards each layer's weights across the GPUs. Set it to the number of GPUs you have for one big model.

10 Quantization (Fit Bigger Models / Go Faster)

Quantization shrinks weights to lower precision so larger models fit in less VRAM. vLLM supports many methods (awq, gptq, fp8, bitsandbytes, gguf, and more).

```
1 # AWQ (4-bit) -- e.g. fit a 70B on a single 80GB GPU
2 vllm serve TheBloke/Llama-2-70B-AWQ --quantization awq
3
```

```

4 # FP8 on modern GPUs (H100/Blackwell): often the single biggest, easiest
   win
5 vllm serve meta-llama/Llama-3.3-70B-Instruct --dtype fp8 --max-model-len
   16384
6
7 # Also store the KV cache in FP8 to save more VRAM
8 vllm serve <model> --dtype fp8 --kv-cache-dtype fp8

```

Rule of thumb: on H100/Blackwell, `--dtype fp8` is the easiest large speed/memory win and needs no separate quantized checkpoint. For older GPUs, use a pre-quantized AWQ/GPTQ model.

11 Serving LoRA Adapters (Ties to Fine-Tuning)

The adapter you trained in the fine-tuning document can be served on top of the base model — and vLLM can hot-swap many adapters on one base model, so you serve several fine-tunes from one GPU.

```

1 vllm serve meta-llama/Llama-3-8B-Instruct \
2   --enable-lora \
3   --lora-modules my_adapter=/path/to/adapter

```

Then request `model="my_adapter"` to use the fine-tuned behavior, or the base name for the original.

12 Docker (Ties to the Docker Document)

There is an official image, so deployment is one command:

```

1 docker run --gpus all --ipc=host -p 8000:8000 \
2   -e HUGGING_FACE_HUB_TOKEN=your_token \
3   vllm/vllm-openai:latest \
4   --model meta-llama/Llama-3-8B-Instruct \
5   --gpu-memory-utilization 0.9

```

`--gpus all` gives the container the GPU; `--ipc=host` is needed for shared memory.

13 Throughput vs Latency (How to Think About Performance)

Two different goals, often in tension:

- **Throughput** — total tokens/second across all users. vLLM's batching maximizes this; great for serving many users or bulk jobs.
- **Latency** — speed for one user, especially TTFT (time to first token). Watch this for interactive chat.

Load-test before trusting a config (e.g. with a tool like Locust) and watch TTFT and requests/second. More concurrency raises throughput but can raise per-request latency.

14 When to Use What

- **Bulk offline jobs** (eval, dataset generation) → the LLM class, one big `generate`.
- **An app / API / chatbot** → `vllm serve` + the OpenAI-compatible client.

- **Model too big for one GPU** → `--tensor-parallel-size`.
- **Not enough VRAM** → quantization (FP8 on new GPUs, AWQ/GPTQ otherwise).
- **Just experimenting on a laptop** → vLLM is overkill; use a tiny model or a hosted API instead.

15 The Checklist for Serving with vLLM

1. Confirm you have a suitable GPU (or rent one).
2. Pick offline (LLM) or online (`vllm serve`).
3. Set `--gpu-memory-utilization` and `--max-model-len` for your hardware.
4. Add `--tensor-parallel-size` if the model spans multiple GPUs.
5. Quantize (FP8/AWQ) if VRAM is tight.
6. Call it with the OpenAI-compatible client.
7. Load-test; watch throughput and TTFT.

16 Practice Task

1. On a GPU (rent one if needed), `pip install vllm`.
2. **Offline:** load a small model with the LLM class and `generate` on a list of 5 prompts; print each completion and its token count.
3. **Online:** run `vllm serve <small-model>` and call it from Python using the OpenAI client pointed at `localhost:8000/v1`.
4. Try `--enable-prefix-caching` and a quantization flag; note any speed/VRAM change.
5. Bonus: run vLLM in Docker, point your LangGraph/LlamaIndex/RAG code at the local endpoint, and (if you did the fine-tuning task) serve your LoRA adapter with `--enable-lora`.